

실시간 피드백을 통한 절차형 VR 교육 시스템

Team VRealize = {강원우, 조현진}

지도교수님 = 이명호 교수님



01

과제 개요

- 연구 배경
- 기존 문제점
- 과제 목표

02

연구 내용

- 시스템 구성도
- 실행 구성도 및 시나리오
- UI 구성

03

구현 방법

- AI Agent
- Object Locator
- 절차적 피드백

CONTENTS

04

시연 영상

- AI Agent
- 절차적 피드백

05

결론

- 기대 효과 및 발전방향

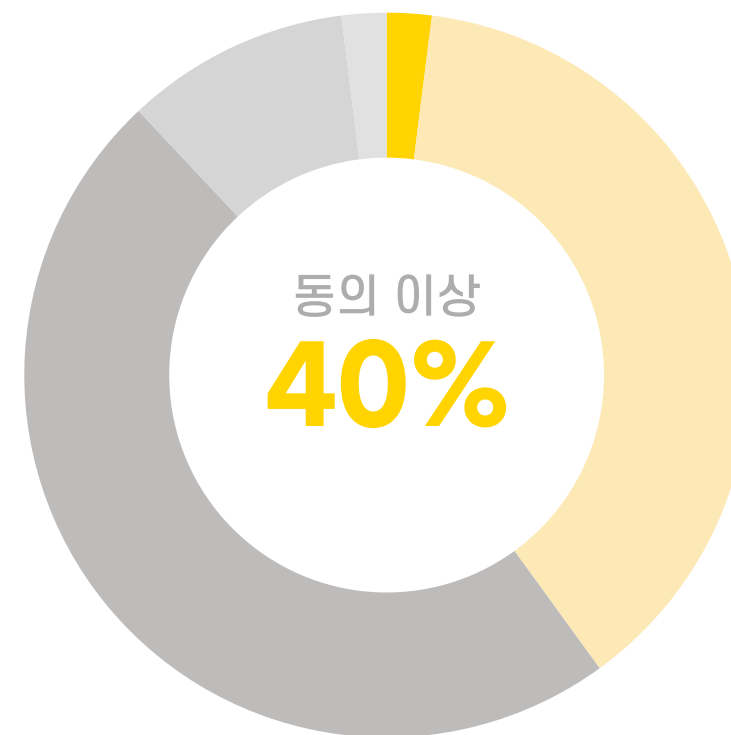
VR 교육 시스템의 등장

- 실습 중심의 학습으로 학습 효율 향상
- 실제 환경과 유사한 경험 제공
- 시간과 공간 제약 없이 반복 학습 가능

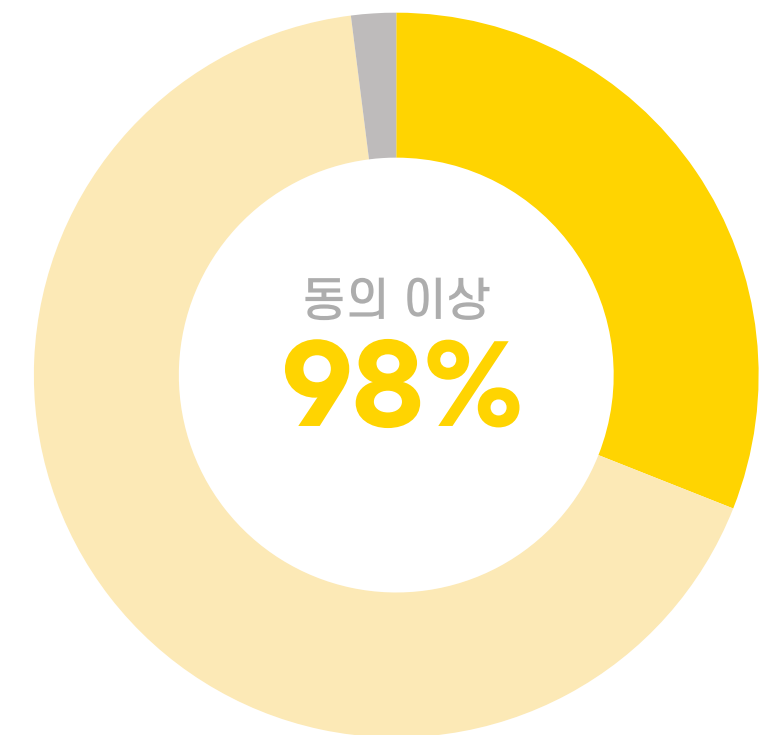


방사선 비상 대응 VR 교육에서 상처제염 과정에 대한 VR 교육 전-후 효과

Q : 상처 제염에 대해 잘 알고 있는가?



[Before Education]



[After Education]



매우 동의



동의



보통



미동의



매우 미동의

기존 VR 교육 뭐가 부족했을까?



피드백 부족

교육 시나리오중 절차적 행동에 대한
실시간 추적 및 피드백 부족



응답 구조의 한계

훈련 중 사용자 질문 발생 시
응답 구조의 한계



단순 지식 전달

단순 지식 전달로 문제 해결 중심의
학습 부족

이를 보완하기 위한 우리의 목표



피드백 부족



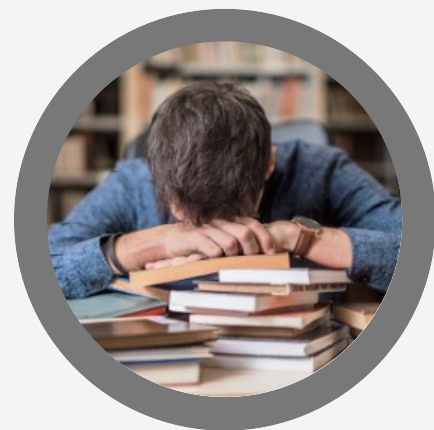
절차적 과정에 대한
추적 및 피드백 강화



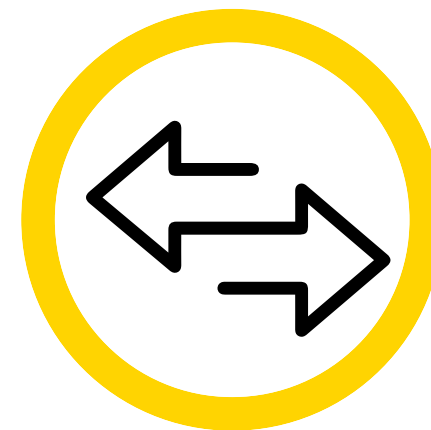
응답 구조의 한계



AI Agent을 통한
실시간 응답 구조 개선



단순 지식 전달



상호작용 확장을 통한
문제 해결 중심 학습

전체 시스템은 어떻게 구성되어 있을까?

시스템 구성도



화재 대피 교육 시스템

본 과제에서는 절차 피드백, AI 상호작용, 실시간 피드백기능을 효과적으로 구현하고 테스트하기 위해 '화재 대피 훈련 시나리오'를 적용 대상으로 선정하였습니다.



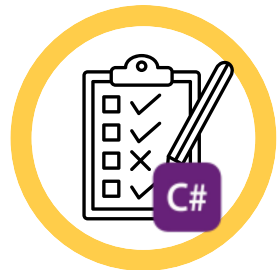
Object Locator

사용자가 특정 오브젝트(예: 소화기, 출입문)를 찾기 어려울 때 위치를 안내하고, 방향을 표시하여 학습을 보조합니다.



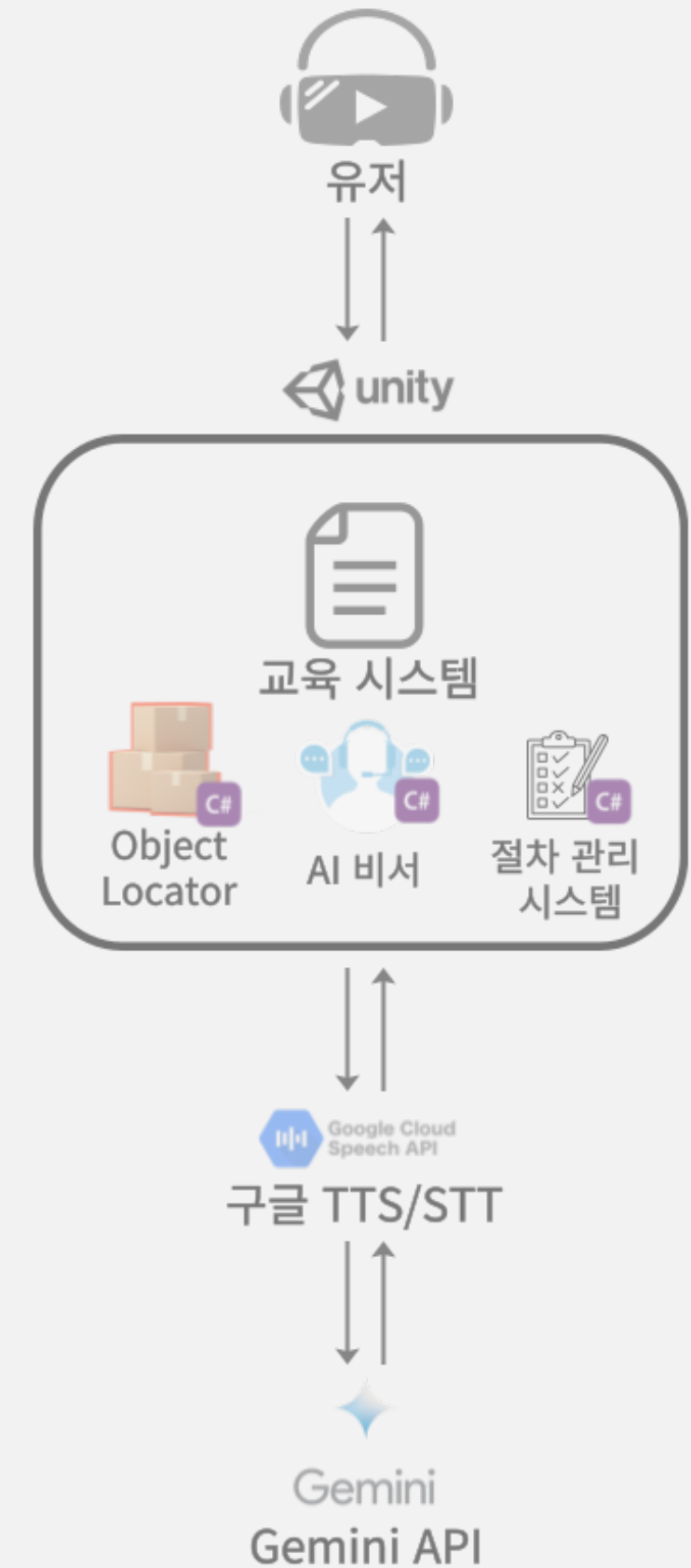
AI Agent

사용자의 음성 명령을 인식하여 즉각적인 응답을 제공합니다.
음성 데이터는 구글 STT를 통해 텍스트로 변환되고, Gemini API를 통해 질문에 대한 답변을 생성합니다.



절차 관리 시스템

학습자가 절차를 올바르게 수행하는지 실시간으로 모니터링하고 피드백을 제공합니다.
예를 들어, 문 손잡이를 만지기 전 온도 확인, 천을 물에 적시는 등의 절차를 추적합니다.



전체 시뮬레이션 구성도



#1. 시작 메뉴

사용자가 학습 모드와 평가 모드를 선택할 수 있는 시작 메뉴입니다. 이 화면에서 대피 훈련 (맵 A,B 중 무작위 선택) 또는 소화기 훈련을 선택하여 훈련을 시작할 수 있습니다.



#2-1. 대피 훈련(A맵)

초기 대피 경로를 따라 이동하는 기본 훈련 시나리오로, 화재 발생 시 기본적인 대피 절차를 학습하는 씬입니다.



#2-2. 대피 훈련(B맵)

사용자가 경로를 암기해 대응력이 떨어지는 문제를 방지하기 위해, 다양한 구조의 맵을 추가하여 여러 시나리오에서 절차를 정확히 수행할 수 있도록 설계한 훈련 씬입니다.

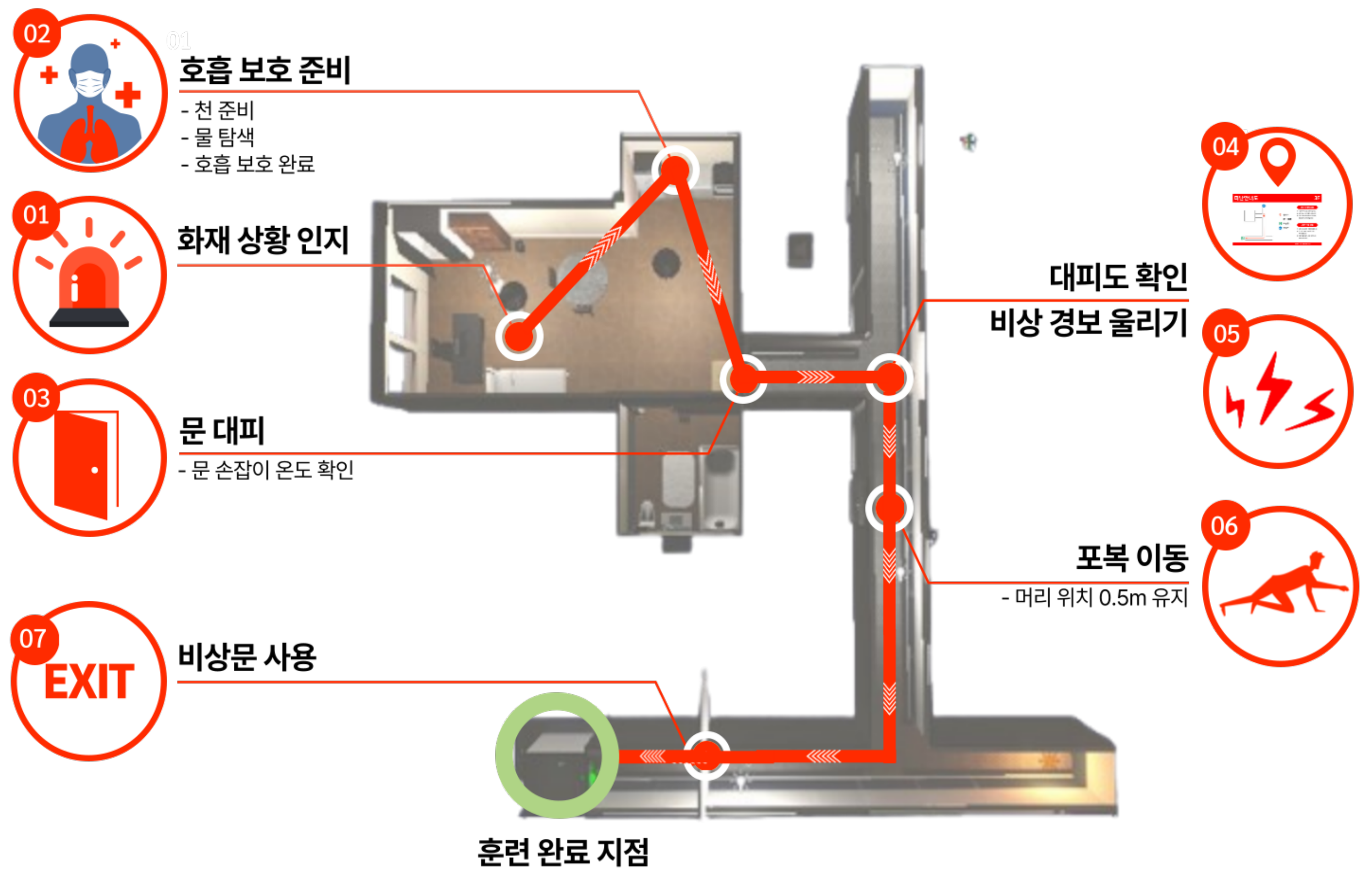


#3. 소화기 훈련

사용자가 소화기를 올바르게 사용하는 방법을 학습하는 씬으로, 소화기 들기, 핀 제거, 노즐 조준, 분사 등의 단계를 포함합니다.

#2-1 대피 훈련(맵 A)

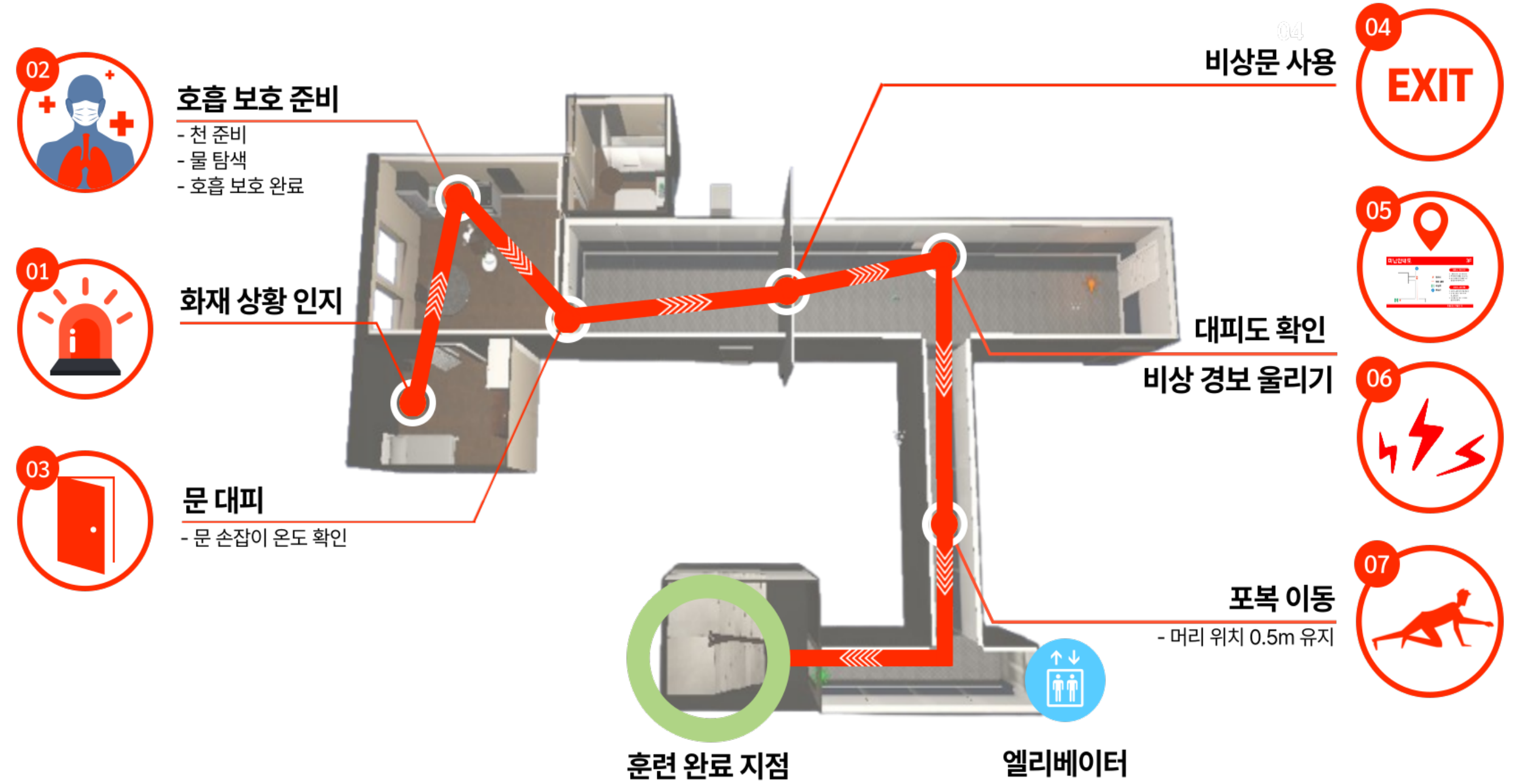
싯 구성도 및 시나리오



#2-2 대피 훈련(맵 B)

실패 구성도 및 시나리오

Particles		
Pause	Restart	Stop
Playback Speed	1.00	
Playback Time	2.73	
Particles	58	
Speed Range	1.0 - 2.9	
Simulate Layers	Nothing	
✓ Resimulate		
Show Bounds		
Show Only Selected		



#3-1 소화기 훈련

씬 구성도 및 구성요소

화재



- 소화기 분사와 상호작용
- 불 VFX 제거 시 훈련 종료

손잡이

- 소화기 그랩 포인트
- 소화기 분사 포인트

안전 핀

- 안전핀을 뽑은 후 분사 진행

노즐

- 분사 VFX 생성 포인트

소화기

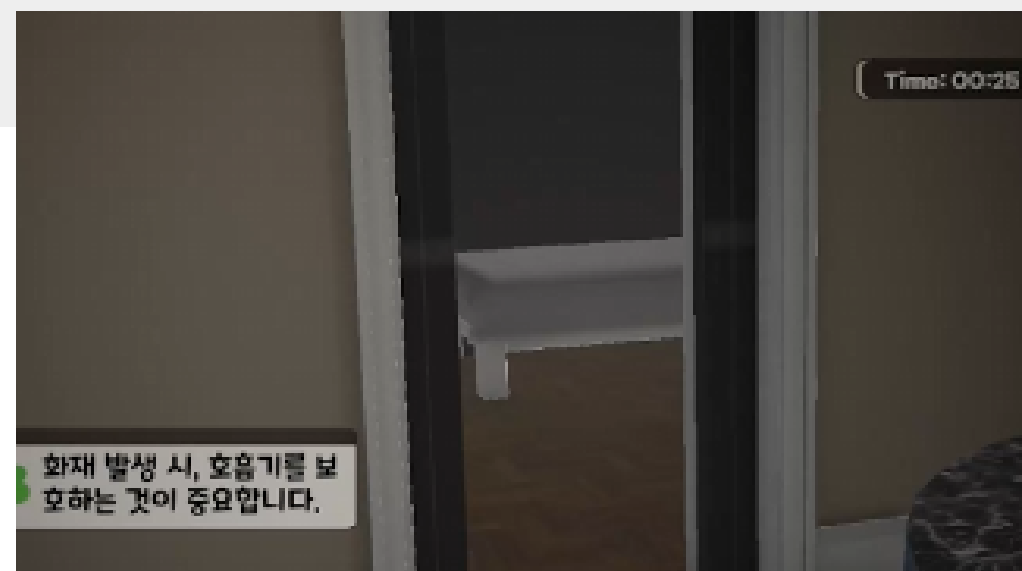
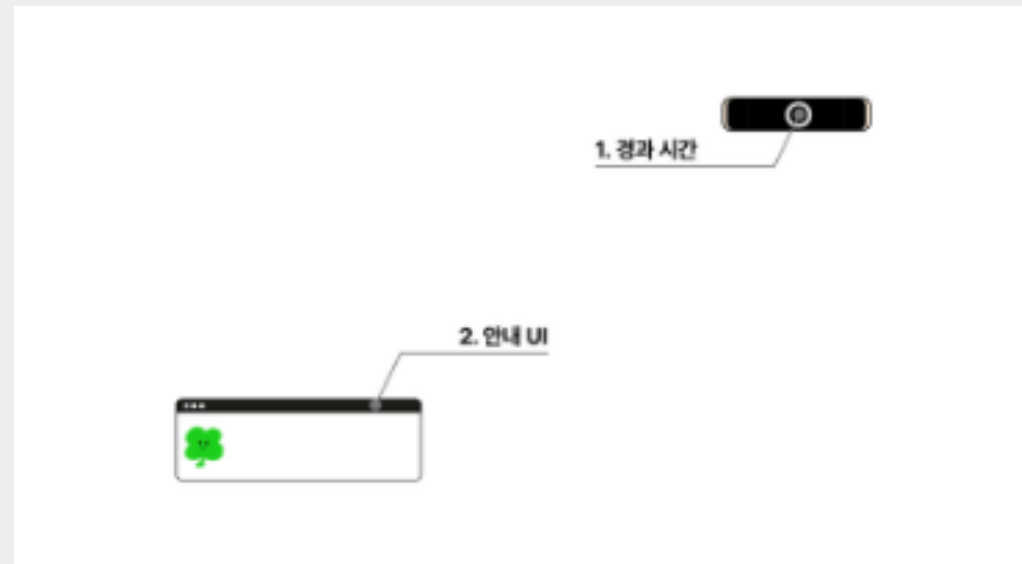


정보 전달의 핵심, UI는 어떻게 구성되어 있을까?

UI 구성

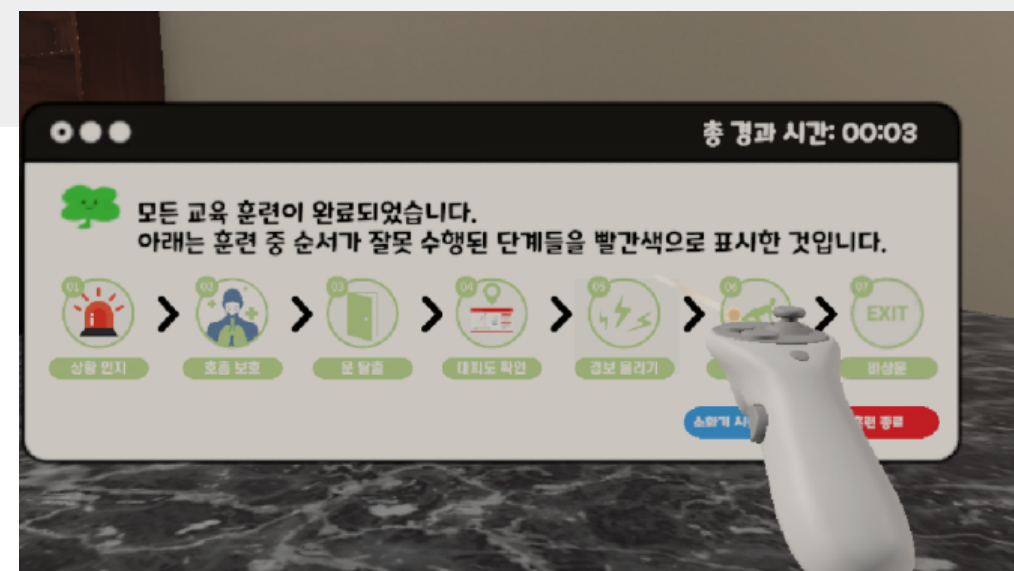
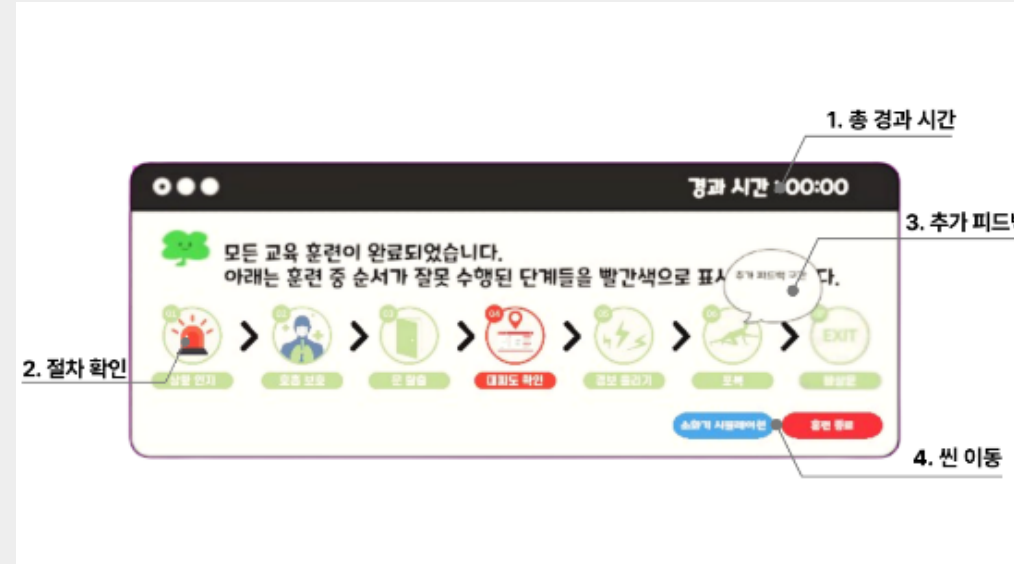
UI 구성

실제 사진



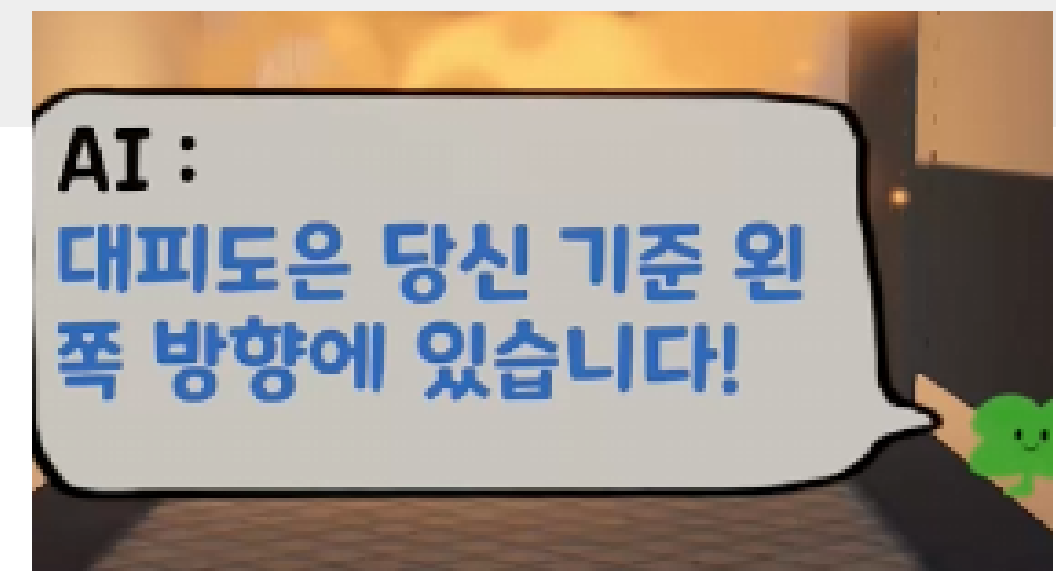
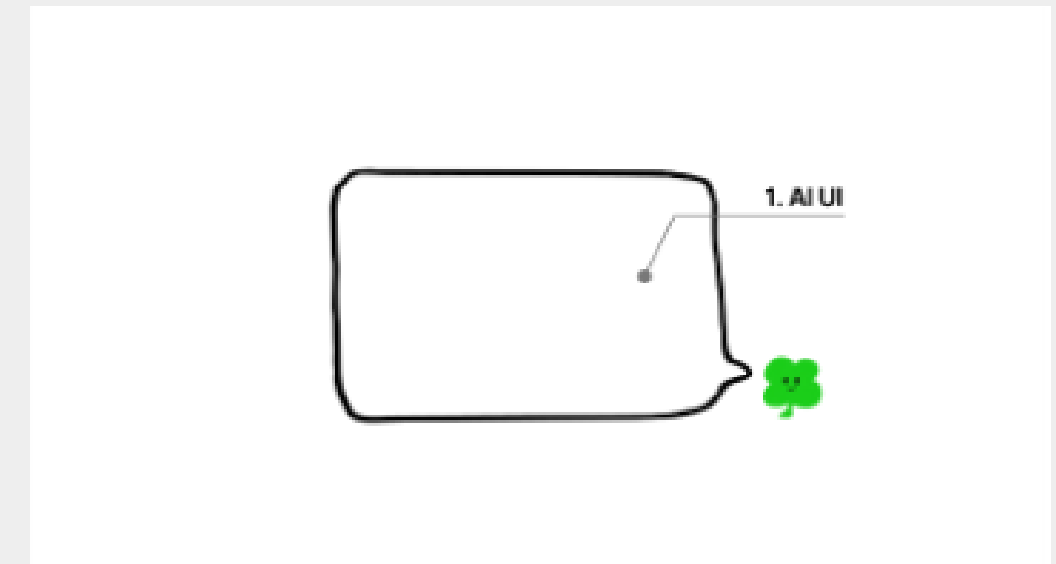
HUD UI

HUD UI는 훈련 프로그램 진행 중 사용자에게 실시간 정보를 제공하며, 효과적인 학습을 지원합니다.



피드백 결과 UI

결과 UI는 훈련 완료 후 사용자의 성과를 시각적으로 정리하여 제공하며, 사용자가 자신의 수행 능력을 평가하고 개선점을 파악할 수 있도록 돕습니다.



AI Agent UI

AI Agent UI는 사용자와 AI 에이전트 간의 상호작용을 지원하며, 실시간 질의응답을 시각적으로 관리합니다.

시스템 구현 방법

SYSTEM OVERVIEW

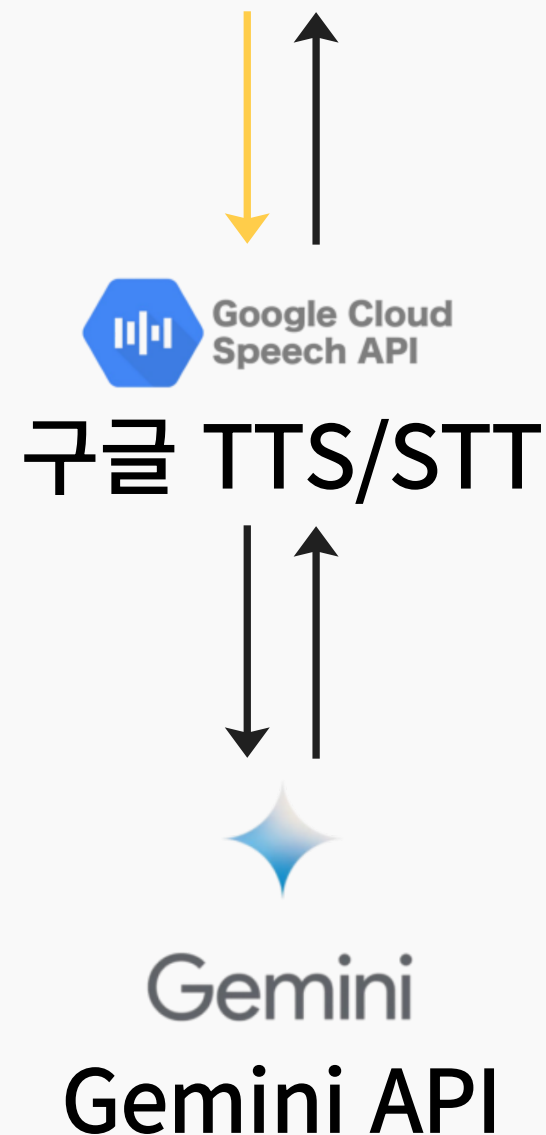
01_ AI Agent

02_ Object Locator

03_ Procedural Feedback



VoiceManager.cs
GoogleSpeechToText.cs
GeminiManager.cs
GoogleTTSTManager.cs



USER TO AI

1. 사용자의 음성 데이터 수집 후, WAV 형식으로 변환하여 바이너리 데이터로 저장.
2. 변환된 데이터는 JSON 형식으로 변환되어 구글 STT API로 전송하여 TEXT로 변환.
3. 전달 받은 TEXT를 AI에게 전달하여 과정 진행.

주요 코드

//샘플 데이터 -> WAV 형식

```
MemoryStream stream = new MemoryStream();  
BinaryWriter writer = new BinaryWriter(stream);
```

```
// WAV 헤더 작성  
writer.Write("RIFF".ToCharArray());  
writer.Write(0); // Placeholder  
writer.Write("WAVE".ToCharArray());  
writer.Write("fmt ".ToCharArray());  
writer.Write(16);  
writer.Write((short)1); // PCM  
writer.Write((short)channels);  
writer.Write(sampleRate);  
writer.Write(sampleRate * channels * 2);  
writer.Write((short)(channels * 2));  
writer.Write((short)16);  
writer.Write("data".ToCharArray());  
writer.Write(samples.Length * 2);  
// 샘플 데이터 쓰기  
foreach (var sample in samples)  
{  
    short intSample = (short)(sample * short.MaxValue);  
    writer.Write(intSample);  
}  
// 파일 크기 수정  
stream.Seek(4, SeekOrigin.Begin);  
writer.Write((int)(stream.Length - 8));  
return stream.ToArray();
```

//STT로 전송하기 위한 JSON 변환

```
private static string ParseGoogleSTTResponse(string json)  
{  
    if (string.IsNullOrEmpty(json))  
    {  
        Debug.LogWarning("[GoogleSpeechToText] 빈 응답");  
        return null;  
    }  
    Debug.Log($"[GoogleSpeechToText] 응답 JSON{json}");  
    if (json.Contains("\"transcript\":"))  
    {  
        int index = json.IndexOf("\"transcript\":") +  
            "\"transcript\".".Length;  
        int start = json.IndexOf("\"", index) + 1;  
        int end = json.IndexOf("\"", start);  
        string transcript = json.Substring(start, end - start);  
        return transcript;  
    }  
    else  
    {  
        return null;  
    }  
}
```


시스템 구현 방법

SYSTEM OVERVIEW

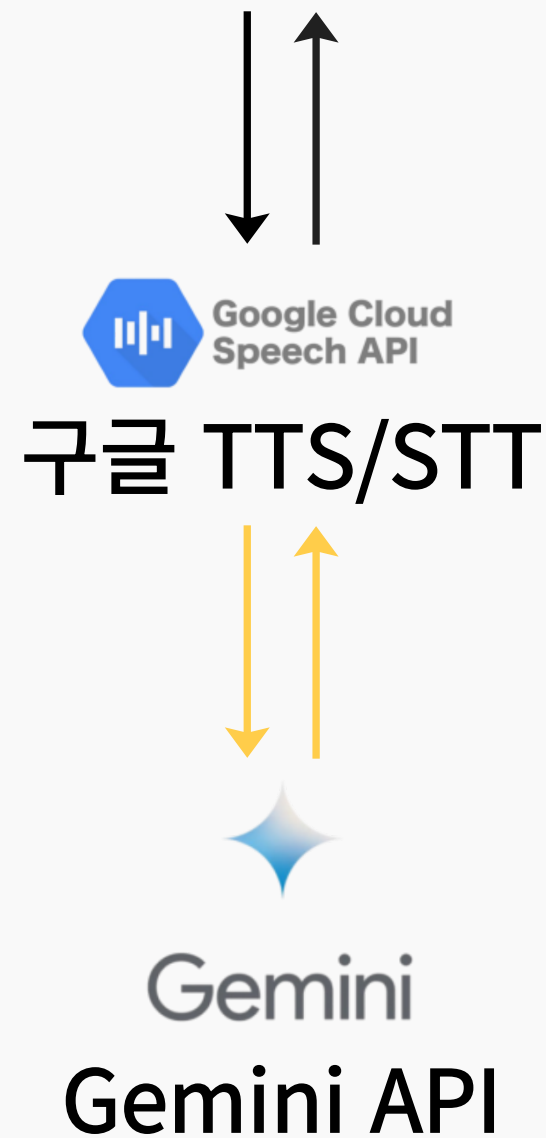
01_ AI Agent

02_ Object Locator

03_ Procedural Feedback



VoiceManager.cs
GoogleSpeechToText.cs
GeminiManager.cs
GoogleTTSTManager.cs



AI 처리

1. STT로 변환된 TEXT는 GEMINI API로 전송, 사전 정의된 컨텍스트를 기반으로 질문의 의도를 파악.
2. AI는 질문에 따라 응답을 생성, 최대 3회의 재시도 로직을 통해 네트워크 오류 방지.
3. 생성된 답변은 JSON 형식으로 반환되며, 후속 단계로 전달.

주요 코드

```
//Gemini AI로 Prompt 전달
public IEnumerator SendPrompt(string question,
    System.Action<string> onResponse)
{
    int retryCount = 0;
    const int maxRetries = 3;

    while (retryCount <= maxRetries)
    {
        using (UnityWebRequest request = BuildRequest(question))
        {
            if (request.result == UnityWebRequest.Result.Success)
            {
                string answer =
                    ParseGeminiResponse(request.downloadHandler.text);
                onResponse?.Invoke(answer);
                yield break;
            }
            else if (request.responseCode == 429)
            {
                retryCount++;
                yield return new WaitForSeconds(2 * retryCount);
            }
            else
            {
                yield break;
            }
        }
    }
}
```

```
//답변을 JSON으로 변환 후 후속 절차로 전송
private string ParseGeminiResponse(string json)
{
    // 간단 파싱: "text" 필드 추출
    if (json.Contains("\"text\":"))
    {
        int index = json.IndexOf("\"text\":") +
            "\"text\":".Length;
        int start = json.IndexOf("\"", index) + 1;
        int end = json.IndexOf("\"", start);
        string text = json.Substring(start, end - start);
        Debug.Log($"[GeminiManager] AI 답변 수신: {text}");
        return text;
    }
    else
    {
        Debug.LogWarning("[GeminiManager] AI 답변 없음");
        return "답변을 가져올 수 없습니다.";
    }
}
```

시스템 구현 방법

SYSTEM OVERVIEW

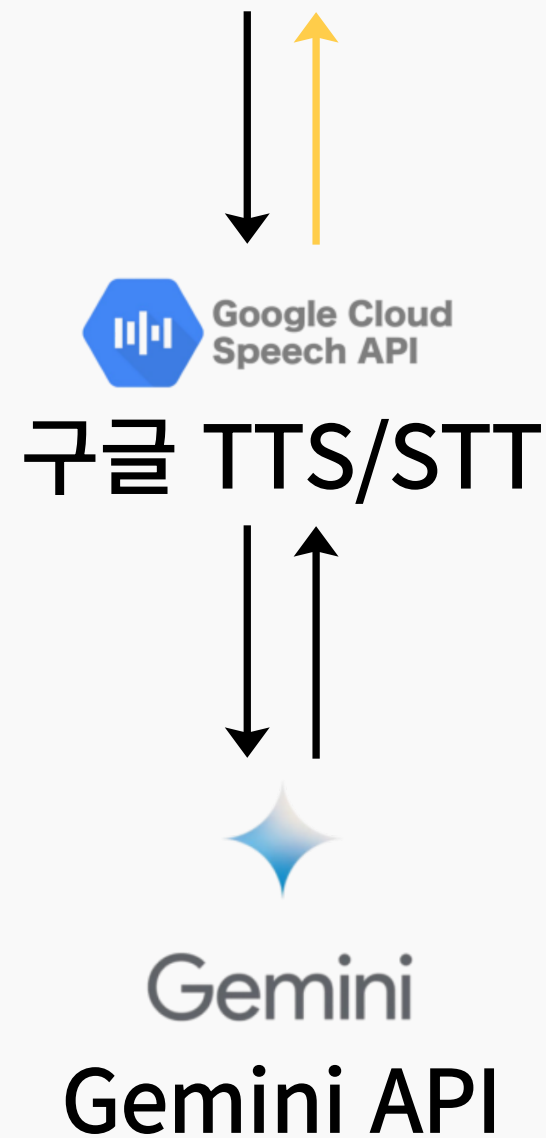
01_ AI Agent

02_ Object Locator

03_ Procedural Feedback



VoiceManager.cs
GoogleSpeechToText.cs
GeminiManager.cs
GoogleTTSManager.cs



AI TO USER

1. 답변 텍스트는 구글 TTS로 전송되며, KO-KR 언어 코드로 설정된 음성으로 변환됩니다.
2. HUD UI를 통해 TEXT, VR 헤드셋을 통해 오디오를 재생.

주요 코드

//전달 받은 답변 텍스트를 TTS 오디오로 변환

```
public IEnumerator Speak(string text)
{
    string url =
        $"https://texttospeech.googleapis.com/v1/text:synthesize?key={ttsApiKey}";

    string safeText = SanitizeText(text);
    var requestBody = new TTSRequest
    {
        input = new Input { text = safeText },
        voice = new Voice(),
        audioConfig = new AudioConfig()
    };
    string jsonBody = JsonUtility.ToJson(requestBody);
    UnityWebRequest request = new UnityWebRequest(url, "POST");
    byte[] bodyRaw = System.Text.Encoding.UTF8.GetBytes(jsonBody);
    request.uploadHandler = new UploadHandlerRaw(bodyRaw);
    request.downloadHandler = new DownloadHandlerBuffer();
    request.SetRequestHeader("Content-Type", "application/json");
    yield return request.SendWebRequest();
    if (request.result == UnityWebRequest.Result.Success)
    {
        string base64Audio =
            ParseTTSResponse(request.downloadHandler.text);
        if (string.IsNullOrEmpty(base64Audio))
        {
            yield break;
        }
        PlayAudio(base64Audio);
    }
}
```

//오디오 파일을 헤드셋에 재생

```
private void PlayAudio(string base64)
{
    byte[] audioBytes = System.Convert.FromBase64String(base64);
    WAV wav = new WAV(audioBytes);
    if (wav.SampleCount == 0)
    {
        return;
    }
    AudioClip audioClip = AudioClip.Create("TTS_Audio",
        wav.SampleCount, 1, wav.Frequency, false);
    audioClip.SetData(wav.LeftChannel, 0);
    GameObject audioObj = new GameObject("TTS_AudioPlayer");
    if (headsetTransform != null)
    {
        audioObj.transform.SetParent(headsetTransform);
        audioObj.transform.localPosition = Vector3.zero;
    }
    AudioSource audioSource =
        audioObj.AddComponent<AudioSource>();
    audioSource.clip = audioClip;
    audioSource.volume = 2.0f;
    audioSource.spatialBlend = 0.0f;
    audioSource.Play();
    Destroy(audioObj, audioClip.length);
}

private string SanitizeText(string input)
{
    if (string.IsNullOrEmpty(input))
    {
        return "";
    }
    return input.Replace("\n", " ").Replace(" ", "").Replace("_",
        "").Replace("#", "");
}
```


시스템 구현 방법

SYSTEM OVERVIEW

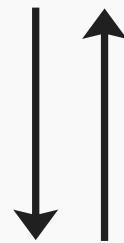
01_ AI Agent

02_ Object Locator

03_ Procedural Feedback



OutlineTarget.cs
ObjectLocator.cs

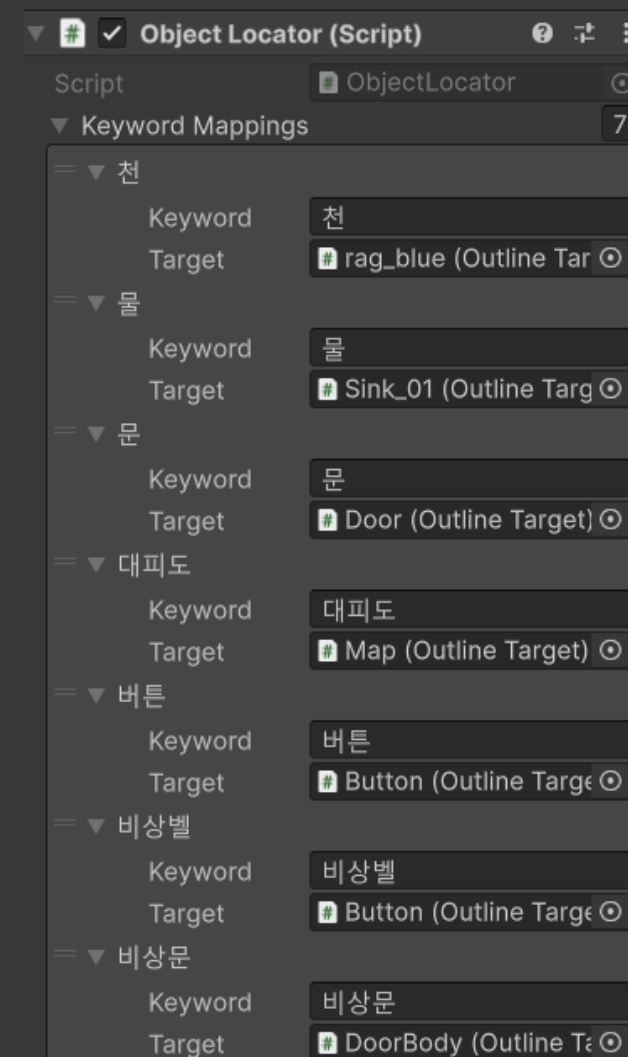


AI Agent

오브젝트 찾기

1. 메서드를 통해 입력 텍스트에서 위치 관련 키워드(예:"어디", "위치")와 오브젝트 명을 탐지.
2. 일치하는 오브젝트가 발견되면 스크립트를 통해 유틸리티를 표시하며, 플레이어 시점에서 오브젝트의 상대 방향을 계산하여 응답.

유니티 스크립트 설정



주요 코드

```
//오브젝트의 상대적 위치 계산
public string GetRelativeDirectionTo(string keyword)
{
    if (string.IsNullOrEmpty(keyword) || playerHeadTransform == null)
        return "주변에 위치해 있습니다";

    // 대상 찾기
    var target = keywordMappings.Find(k => k.keyword == keyword)?.target;
    if (target == null) return "주변에 위치해 있습니다";
    Vector3 toTarget = (target.transform.position - playerHeadTransform.position).normalized;
    Vector3 forward = playerHeadTransform.forward;
    // 평면 방향 (수평 판단만)
    toTarget.y = 0;
    forward.y = 0;
    float angle = Vector3.SignedAngle(forward, toTarget, Vector3.up);
    if (angle >= -45f && angle < 45f) return "앞";
    if (angle >= 45f && angle < 135f) return "오른쪽";
    if (angle >= -135f && angle < -45f) return "왼쪽";
    return "뒤";
}
```

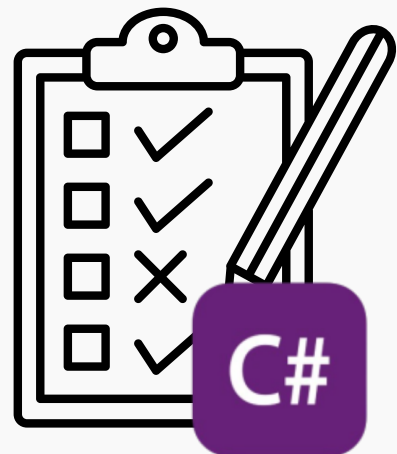
시스템 구현 방법

SYSTEM OVERVIEW

01_ AI Agent

02_ Object Locator

03_ Procedural Feedback



SequenceManager.cs
EndUI.cs

절차 피드백 구현

1. 훈련 도중 사용자가 잘못된 절차를 수행하면, 즉시 HUD UI를 통해 경고 메시지를 시각적으로 전달.
2. 이전 단계가 완료되지 않으면 다음 단계로 진행할 수 없도록 락 시스템 구현.
3. 훈련 종료 후에는 결과 UI를 통해 사용자의 전체 절차 수행 내역을 시각적으로 요약.

주요 코드

//시퀀스 절차 관리

```
private bool[] sequenceErrors = new bool[7]; // 7단계: 상황 인지, 호흡 보호, 문 탈출, 대피도 확인, 경보 울리기, 포복, 비상문 사용
private bool[] sequenceCompleted = new bool[7]; // 각 단계 완료 여부
private int currentStep = 0; // 현재 진행 중인 단계 (0-based index)
```

// 특정 단계에서 순서 오류를 기록

```
public void RecordSequenceError(int stepIndex)
{
    sequenceErrors[stepIndex] = true;
}
```

// 단계 완료 기록

```
public void CompleteStep(int stepIndex)
{
    if (stepIndex < 0 || stepIndex >= sequenceCompleted.Length) return;
    sequenceCompleted[stepIndex] = true;
    if (stepIndex == currentStep)
    {
        currentStep++;
    }
}
```

//결과 UI에서 시퀀스 에러를 확인하여 UI 출력

```
private void DisplaySequenceStatus()
{
    for (int i = 0; i < stepImages.Length; i++)
    {
        if (SequenceManager.Instance.HasSequenceError(i))
        {
            stepImages[i].texture = incorrectTextures[i];
        }
    }
}
```

결론

기대효과 및 발전방향

본 연구는 절차 중심의 훈련 교육에서 발생하는 기존 시스템의 한계를 극복하기 위해, 다양한 시스템을 통합한 VR 화재 대피 및 소화기 훈련 시스템을 개발하였다. 개발된 시스템은 다음과 같은 기여를 할 것으로 기대 된다.



사용자의 훈련 효율 증가

AI 응답과 실시간 피드백을 통해 학습 몰입도와 자율성이 높아지면서, 사용자는 이전보다 더 많은 학습 효과를 얻을 수 있습니다. 또한, 학습 과정 자체를 분석하고 반응할 수 있어 향후 인터랙티브 교육 시스템으로의 발전 가능성도 기대할 수 있습니다.



훈련 콘텐츠 다양화

화재 대피 외에도 지진, 산업 안전, 교통사고 대처 등 다양한 재난 상황에 적용 가능한 시스템을 구축함으로써 VR 기반 종합 재난 대응 교육 플랫폼으로 확장할 수 있다.



AI 상호작용 확장

AI에게 질의응답이나 문제 위치 안내뿐만 아니라, 올바른 대피 절차를 학습시켜 사용자의 동작을 실시간으로 추적하고 평가하는 등 다양한 확장이 가능합니다.

시연 영상

AI Agent



시연 영상

절차형 피드백

